AI Engineering and Claude Architect · Part IV, Java

AI in Java, Lesson 1

Claude in Spring Boot: Client, Config, Errors

The service called Claude from inside a controller, with a new client per request and the key in a property file. It worked in the demo. In production it ran out of connections on Monday, returned a raw stack trace with a key fragment on Tuesday, and retried a bad request forever on Wednesday.

Module	AI08 of 13
Lesson	1 of 8
Prerequisites	AI03 L1, AI07 L1, and Spring Boot
Exam relevance	Language-specific work is out of scope. Stop reasons and errors support TS 1.1 and TS 2.2
Reading time	~35 minutes

Contents

1. Claude in Spring Boot

Lesson 1 of 8 · AI in Java

Claude in Spring Boot

Key idea

Treat Claude like any other remote dependency: one shared client, validated configuration, explicit timeouts, and failures translated into what your application should do. Keep the SDK in one adapter behind your own port. **Then test that adapter by pointing the real SDK at a local fake server, so you can see the exact request it sends, without a key.**

> *Before: the Messages API (AI03 L1), the port pattern (AI07 L1). Teaches: client lifecycle, configuration, error translation, safe API errors, and testing against a fake API. Used by: all of AI08.*

The project, and a version warning

`code/ai08-java` is a real Spring Boot service that this whole module builds on. **Its 26 tests pass, in about 43 seconds, with no API key.** They use real embedded Tomcat, real Kafka, real MCP over HTTP, and the real Anthropic SDK talking real HTTP to a local fake.

What that can't prove: the fake's replies are JSON that I wrote from the API reference. So nothing here shows that the real API agrees, or anything about model behavior.

Component	Version	Note
Java	21	
Spring Boot	3.5.16	**Open-source support ended on 30 June 2026**
`anthropic-java`	3.2.0	Directly, without a framework
MCP Java SDK	2.1.0	Latest stable
spring-kafka	Boot-managed	Kafka client 3.9.2

Boot 3.5 is past open-source support

Per endoflife.date, Boot 3.5's open-source support ended in June 2026, and commercial support runs to 2032. Boot 4.0 has open-source support to December 2026, and 4.1 to July 2027. This course stays on Boot 3 by earlier decision, and most of this code should move across, though I haven't tested it on Boot 4. For a new service today, start on a supported line. Spring AI follows the same split: 1.1.x supports Boot 3.4 and 3.5, and 2.0.x supports Boot 4.0 and 4.1.

The hook: see what's really sent

The real SDK, sending to a local server, which records the body. Real output:

```
request path: /v1/messages
request body: {"max_tokens":16000,
  "messages":[{"content":"Hi","role":"user"}],
  "model":"claude-opus-5-5",
  "output_config":{"effort":"medium"},
  "system":[{"text":"You are a support agent.","type":"text",
             "cache_control":{"type":"ephemeral"}}],
  "tools":[{"input_schema":{"type":"object","properties":{"order_id":{"type":"string"}},
            "required":["order_id"],"additionalProperties":false},
            "name":"lookup_order","description":"Looks up one order.","strict":true}]}
```

Line breaks are added. **Check what isn't there:** no `temperature`, no `tool_choice`, no `thinking` budget. Those would return a 400 on Opus 5.5, so a test asserts their absence (AI07 L4). That's a test worth having, and it needed no key.

Intuition: a supplier contract

You wouldn't let every employee phone a critical supplier with a personal arrangement. There's one account, one contact route, an agreed response time, and clear rules for "out of stock" versus "wrong order". **The adapter is that account.**

Mechanism: the four parts

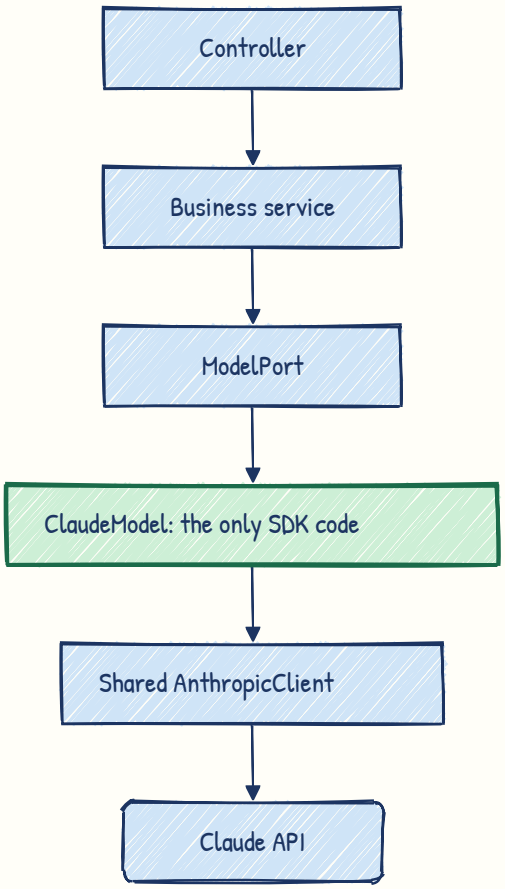


Figure 1. How a request flows through the service. Only one class knows about the SDK, and errors are translated on the way back.

1. Validated configuration

```
@Validated
@ConfigurationProperties("course.claude")
public record ClaudeProperties(
        @NotBlank String model,
        @Min(1) @Max(128000) long maxTokens,
        @NotNull Duration timeout,
```

A bad value now **stops the deploy**, rather than the first customer. The key comes from the environment or a secret store, as `COURSE_CLAUDE_API_KEY`. It's never in a file you commit.

Setting	Used here	Why
`model`	`claude-opus-5-5`	Exact ID, in config so that it can change without a code change
`max-tokens`	16000	Non-streaming requests stay under SDK timeouts
`timeout`	120s	Tune it to your use. Note retries multiply it
`max-retries`	2	The SDK's default
`effort`	`medium`	Set it explicitly, since it's a cost and quality decision

Worst-case wall clock is `timeout × (retries + 1)`, so 120 seconds with 2 retries can take **6 minutes**. Make sure your own HTTP timeouts agree.

2. One shared client

```
@Bean(destroyMethod = "close")
@ConditionalOnProperty("course.claude.api-key")
AnthropicClient anthropicClient(ClaudeProperties properties) {
```

Each client holds its own HTTP connection pool, so a client per request wastes connections. And **with no key set, the app starts on a scripted model**, which says so in its answers. That keeps local development and tests simple.

3. The adapter

Port types carry no SDK classes, with one deliberate exception. `Turn.raw` holds the SDK's whole message as an opaque `Object`, because the next request must replay it unchanged, including thinking blocks (AI08 L3).

4. Translating errors

Catch the most specific first. One broad catch would lose the difference between retryable and permanent.

```
} catch (RateLimitException e) {
    throw new ModelException(ModelException.Kind.RATE_LIMITED, "Rate limited", e);
} catch (InternalServerException e) {
```

Real output, against the fake server:

```
400 -> BAD_REQUEST  retryable=false requests=1
401 -> AUTH         retryable=false requests=1
404 -> NOT_FOUND    retryable=false requests=1
429 -> RATE_LIMITED retryable=true  requests=3
529 -> OVERLOADED   retryable=true  requests=3
429 then 529 then 200: answer=Done requests=3
```

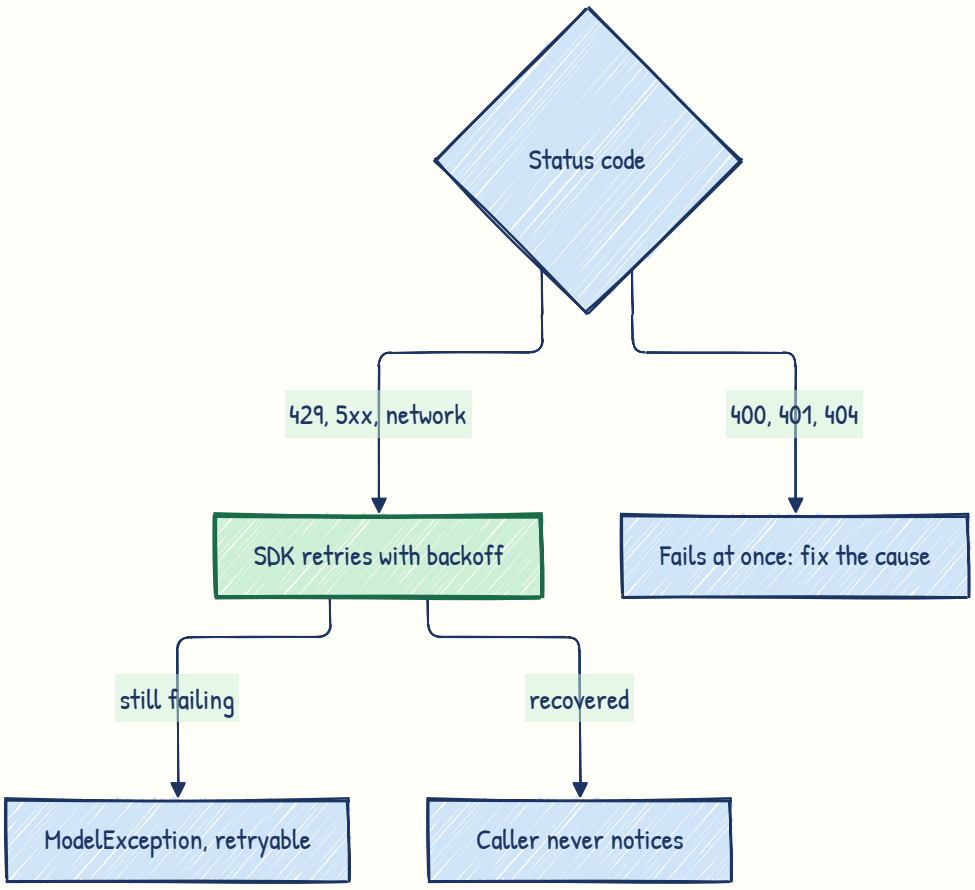


Figure 2. Which failures get retried. The SDK retries transient ones by itself, so your code only sees them after its attempts run out. Permanent ones arrive at once.

Don't add your own retry around this without thinking. Retrying around the SDK's retries multiplies the attempts. If a queue also retries (AI08 L6), that's a third layer.

Safe API errors

The caller gets what they can act on, and the logs keep the detail. Real output, where the exception message held a fake key:

```
rate limited: 429 {"title":"Too Many Requests","status":429,
  "detail":"The assistant is busy. Please try again shortly.","kind":"RATE_LIMITED","retryable":true}
bad key: 500 {"title":"Internal Server Error","status":500,
  "detail":"The assistant is unavailable.","kind":"AUTH","retryable":false}
```

Output is trimmed. Note that a bad key is **our** fault, so the caller gets a 500, not a 401. The test asserts that `sk-ant` never appears in the body.

What should alert

Kind	Meaning	Who acts
`AUTH`	Bad or rotated key	**Page someone**
`NOT_FOUND`	Wrong or retired model	**Page someone**
`BAD_REQUEST`	Our code or a new model's rules	Ticket, urgent after a deploy
`RATE_LIMITED`	Limits reached	Alert on rate, then capacity work
`OVERLOADED`	Provider busy	Alert on rate. Check status page

Why the obvious fixes fall short

Tempting move	Why it fails	Better
`new AnthropicOkHttpClient` per request	Pools pile up	One bean
Catch `Exception`	Retryable and permanent look alike	Typed chain
Return the exception message	Leaks internals	`ProblemDetail`, with safe text
`@Retryable` around every call	Multiplies SDK retries	Let the SDK retry, and decide by kind
Model name in code	A code change per model	Configuration
Mock `AnthropicClient` in tests	Doesn't test the JSON	Point the real SDK at a fake server

How Anthropic and the field approach it

Point	Source
SDKs retry 429 and 5xx automatically, with `max_retries` default 2	Claude API reference
Java exception classes such as `RateLimitException` and `InternalServerException`	Same
Catch most-specific first, since a broad catch loses retryable vs not	Same
`.model(String)` works for any model, and typed constants lag launches	Same

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: SDK calls in controllers.

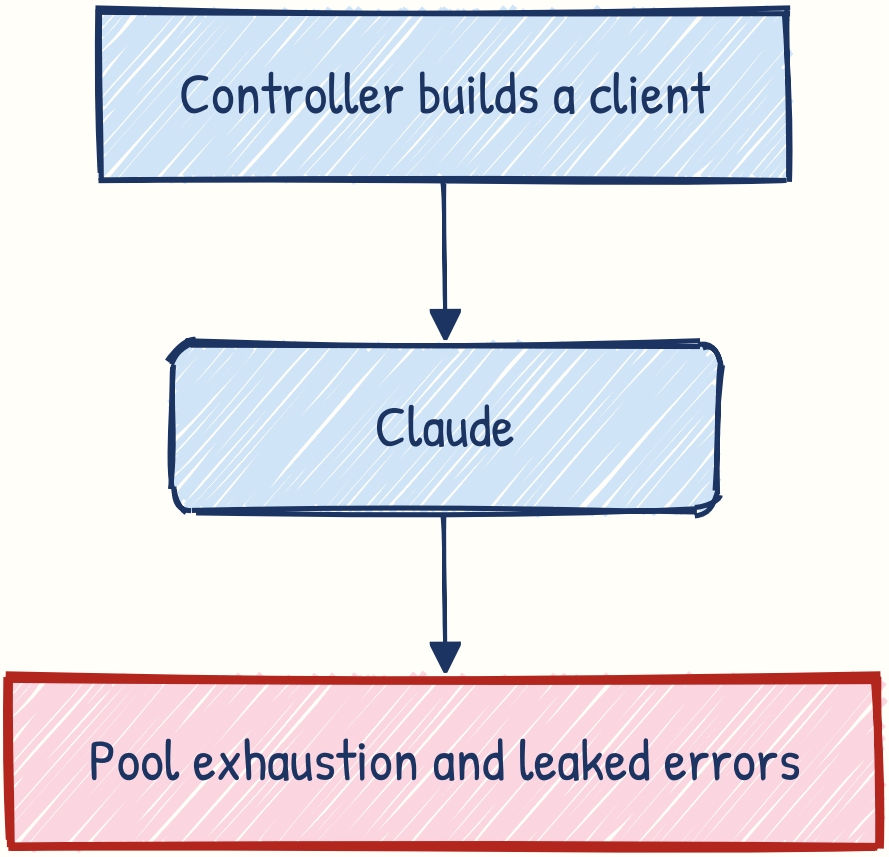


Figure 3. Baseline. Controllers build their own clients and leak failures straight to callers.

Improved: a shared adapter. New problem: one bad deploy or quota problem hits every feature at once, and one noisy feature can starve the others.

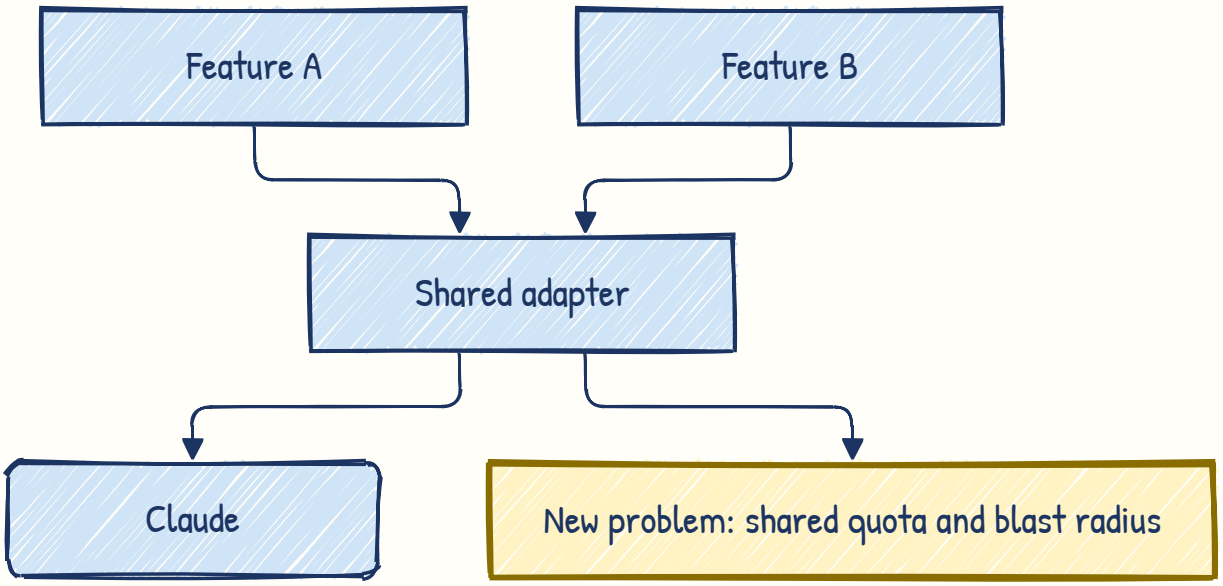


Figure 4. Improved. One adapter tidies the code, but all features share one quota and one failure.

Production: an internal gateway layer.

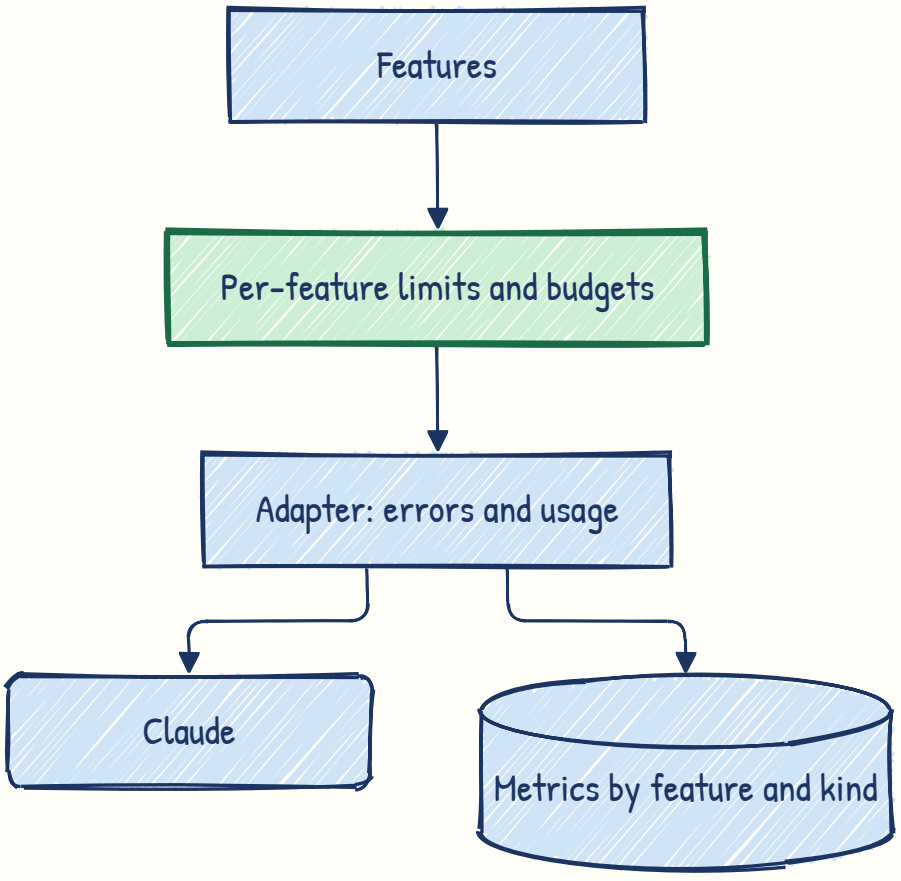


Figure 5. Production. Per-feature limits sit in front of the adapter, usage is metered by feature, and failures are classified for alerting.

Component	Why it exists	How it communicates
Per-feature limits	One feature can't starve the rest	Semaphore or token bucket, in-process or shared
Usage metering	Cost by feature	Micrometer counters from `Turn` usage
Workspaces	Separate keys and limits per team	Claude Console. Separate batch work from live traffic
Secret store	Rotation with no redeploy	Injected as an environment variable

Failures

Failure	Containment
Key rotated badly	Alert on `AUTH`, and keep the old key valid during rollout
New model rejects a parameter	Nightly live test, and a canary
Long hang	Explicit timeout. Check for time spent in retries
Quota exhausted	Per-feature budget, and shed low-priority work first

Trade-offs

Decision	Gain	Cost
Direct SDK	Newest features, clear requests	You write some plumbing
Own port	Testable, replaceable	Must allow provider features through
Opaque `raw`	Replays correctly	Port isn't perfectly pure
Fake API server	Tests JSON, retries and errors with no key	Its replies may drift from the real API

When simpler is fine: a batch script doesn't need a port.

Common mistakes

1. **A client per request.** Share one.
2. **The key in a committed file.** Use the environment or a secret store.
3. **A single broad catch.** Use typed exceptions.
4. **Retrying on top of SDK retries.** Attempts multiply.
5. **Leaking exception text to callers.** Log it instead.
6. **Forgetting that retries multiply timeouts.** 120 seconds can become 6 minutes.
7. **Assuming a fake server proves API compatibility.** Keep a nightly live test.
8. **Adopting an out-of-support Boot line for new work.** Check first.

What to say in an interview

"I'd treat Claude as a remote dependency: one client bean, validated properties, and a typed error chain mapped to retryable and permanent kinds. Only one adapter imports the SDK. I'd test it by pointing the real SDK at a local fake, and asserting on the JSON, because that catches a forbidden parameter. A nightly call to the real API covers what a fake can't."

Certification lens

How this lesson maps to CCAR-F

Implementation details aren't tested. Concepts are: transient versus permanent failures (**TS 2.2**), and stop reasons (**TS 1.1**).

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Debugging · Medium. Your service makes one model call with `timeout` 60s and `maxRetries` 2. A gateway in front times out at 90s, and users see errors during a brief overload, though the call would have succeeded. Why, and what are two fixes?

A Worst case is 60 × 3 = 180 seconds, which outlasts the gateway's 90. Fixes: lower the client timeout or retries so that the total is under 90, or stream, so the gateway sees early bytes, or move long work to a queue (AI08 L6).

Q Q2 · Code analysis · Medium. A teammate wraps the adapter in `@Retryable(maxAttempts = 3)`, and a Kafka consumer above it retries 3 times. The SDK has `maxRetries` 2. How many API calls could one failing event make?

A 3 Kafka attempts × 3 `@Retryable` attempts × 3 SDK attempts = **27 calls**. During an overload, that makes things worse. Retry at one layer, or two with clear purpose, and use a circuit breaker.

Q Q3 · Best decision · Medium. A test mocks `client.messages().create()` and returns a canned `Message`. What can't this catch that a fake HTTP server can? What can't either catch?

A The mock skips request serialization, so it misses a forbidden field, a wrong header or path, real SDK retries, and real error mapping. Neither can catch the real API rejecting something, a changed response, or model behavior, so keep a small live test.

Revision - Lesson 1

→ Treat Claude as a remote dependency, with one client bean.

→ Validate configuration at startup, and keep keys out of files.

→ Only one adapter imports the SDK. The port uses your own types.

→ Typed exceptions, most specific first, mapped to retryable or permanent.

→ SDK retries 429 and 5xx by default, so don't stack retries blindly.

→ Worst-case time is timeout times attempts.

→ Log details, and send callers safe problem details.

→ Alert on AUTH and NOT_FOUND as urgent.

→ Point the real SDK at a fake server to test JSON, retries and errors.

→ A fake can't prove what the real API does, so run a nightly live test.

→ Boot 3.5 is past open-source support. Check lines before starting new work.

Your notes